MATT'S READING GROUP · APRIL 2026

Recursive Language Models

Letting the model decide how to look at its own context

MOTIVATION

Inference-Time Scaling Has Three Acts

ACT I · 2022

Chain of Thought

Test-time compute **within a single call**

```
query
|
think…
|
answer
```

ACT II · 2023

ReAct / Agents

Test-time compute **across tool calls**

```
call → tool
|
call → tool
|
call → tool
|
answer
```

ACT III · 2025

RLMs

Test-time compute **over context decomposition**

```
peek ctx
|
rlm(chunk₁)
|
rlm(chunk₂)
|
aggregate
```

THE PROBLEM

Long Contexts Degrade in Practice

What benchmarks say

RULER / Needle-in-Haystack

"Is the answer somewhere in the context?"

~Saturated ✓

What users experience

"My Claude Code session gets noticeably worse after an hour — even when the answer is right there."

CONTEXT ROT

The model **technically sees the tokens**, but attention becomes diffuse over long sequences — performance degrades even when the answer is in the window.

Why it's hard to study

Standard evals can't benchmark "conversations feel worse over time." We need distributional queries at scale — which is what OOLONG was built for.

THE PROBLEM

All Existing Fixes Impose Structure from Outside

RETRIEVAL

RAG

Human builds an external index. Chunking and retrieval strategy designed up front — before the query is known.

MEMORY

MemGPT · MemWalker

Human designs a tree or paging structure. Model navigates it — it doesn't define it.

SUMMARIZATION

Agent Scaffolds

Human decides when and how to compress context — at fixed intervals, by fixed rules.

COMMON PATTERN

A human decides the decomposition strategy. The same mistake we made before CoT — hand-designing the structure instead of letting the model find it.

PART II

The Reframe

THE REFRAME

Context Decomposition Should Be the Model's Decision

AGENTS TODAY

Problem-centric decomposition

Break the **task** into subtasks → each subtask queries whatever context it needs

Structure lives in the task planner — a human-designed scaffold

RLMS

Context-centric decomposition

Break the **context** into pieces → model decides how to query programmatically

If the model should decide *how to reason*, why not *how to look?*

THE REFRAME

An RLM Is a Drop-In Replacement for an LM Call

Standard LM

```
# str → str
response = lm.completion(messages)
```

With an RLM

```
# same type signature: str → str
response = rlm.completion(messages)
```

Not a framework

The caller writes the same call. The RLM decides internally whether to peek, chunk, recurse, or pass through — the decomposition strategy is invisible to the user.

Backward-compatible by design

Drop into any existing pipeline with a one-line swap. No new SDK, no new orchestration layer.

PART III

A Full Trajectory

TRAJECTORY WALKTHROUGH

The Root LM Sees Only the Query and a REPL

The query

"For users U1 … U20 — how many rows in this dataset have labels X, Y, Z?"

Dataset: ~3,000 rows · ~132k tokens

What the root LM (depth=0) receives

```
# Root LM scope
query = "For users U1…U20, count labels"
ctx   = load_context()   # ~132k tokens
# ctx is a variable — not raw tokens
# Root LM never sees it directly
```

The key design choices

- Context is a **Python variable**, not tokens in the prompt
- Model interacts by **writing code cells** in a REPL
- Can invoke **rlm()** recursively on any string
- Returns a final answer via `FINAL(answer)`

That's the whole mechanism. Everything else emerges.

TRAJECTORY WALKTHROUGH

STEP 1

Peek at the Data Format

Root LM writes

```
# Sample the first 2000 chars
print(ctx[:2000])
```

Output

```
user_id,timestamp,label,text
U1,2024-01-03,positive,"great product"
U3,2024-01-03,negative,"disappointing"
U1,2024-01-04,neutral,"it arrived"
...
```

What the model learns

- CSV format with a `user_id` column
- Labels are: positive, negative, neutral
- Can now plan a regex-based strategy

Inspects ~1.5% of the context

TRAJECTORY WALKTHROUGH

STEP 2

Grep for Relevant Rows

Root LM writes

```
user_ids = ["U1", "U2", ..., "U20"]
rows = ctx.split("\n")

filtered = [
  row for row in rows
  if any(uid in row for uid in user_ids)
]
print(len(filtered))
```

```
>> 847 matching rows
```

3,000 → 847 rows

A 72% context reduction — before a single LM token is spent reading content.

Note

This is plain Python string processing — free, deterministic, milliseconds. The LM didn't read the full context. It filtered programmatically.

TRAJECTORY WALKTHROUGH

STEP 3

Partition Filtered Rows into Chunks

Root LM writes

```
CHUNK_SIZE = 150  # rows per chunk

chunks = [
  filtered[i : i + CHUNK_SIZE]
  for i in range(0, len(filtered), CHUNK_SIZE)
]
print(len(chunks), "chunks")
```

```
>> 6 chunks
```

847 rows → 6 chunks

150 chunk 1	**150** chunk 2	**150** chunk 3
150 chunk 4	**150** chunk 5	**97** chunk 6

Each chunk fits comfortably in a recursive sub-call's context window.

TRAJECTORY WALKTHROUGH

STEP 4

Spawn Recursive Sub-Calls

Root LM writes

```
results = []
for i, chunk in enumerate(chunks):
  sub_q = (
    f"Count label occurrences per user"
    f" in this data:\n{chunk}"
  )
  # Recursive call — depth = 1
  r = rlm.completion([
    {"role": "user", "content": sub_q}
  ])
  results.append(r)
```

The recursive call

- **depth=1** sub-LM gets one chunk (~15k tokens)
- Sub-LM can also peek, grep, or recurse
- Returns structured counts per user / label

6 sub-calls — one per chunk

TRAJECTORY WALKTHROUGH

STEP 5

Aggregate and Return

Root LM writes

```
counts = {}
for r in results:
  partial = parse_counts(r)
  for user, labels in partial.items():
    if user not in counts:
      counts[user] = {}
    for label, n in labels.items():
      counts[user][label] = (
        counts[user].get(label, 0) + n
      )

FINAL(counts)
```

FINAL() exits the REPL

The aggregated dict becomes the `rlm.completion()` return value.

The full strategy — invented at inference time

1. PEEK → understand format
2. GREP → 3k rows → 847
3. PARTITION → 6 chunks
4. MAP → 6 recursive calls
5. AGGREGATE → FINAL()

None of this was hardcoded.

METHOD

Four Strategies Emerge Without Being Engineered

PEEK

Sample before committing

Reads a small prefix to infer format and schema before planning the full strategy.

```
ctx[:2000]
```

GREP

Filter by pattern

Uses regex or substring matching to extract only the rows relevant to the query — a free text index.

```
re.findall(...)
```

PARTITION + MAP

Divide and recurse

Splits the context and spawns recursive sub-calls to process each chunk independently.

```
rlm(chunk_i)
```

SUMMARIZE

Compress before reasoning

Distills sections into shorter representations, enabling multi-hop retrieval over large corpora.

```
rlm("summarize:...")
```

These behaviors fall out of the REPL design — the model discovers them, not the engineer.

PART IV

Results

OOLONG-PAIRS · INFORMATION-DENSE TASK

Base GPT-5 · F1

0.04%

RLM(GPT-5) · F1

58%

An emergent capability —
the base model essentially fails

What OOLONG-Pairs tests

Find pairs of entries with semantically opposite labels — requires processing every line with deep understanding. Roughly quadratic effort relative to context length.

Also on standard OOLONG

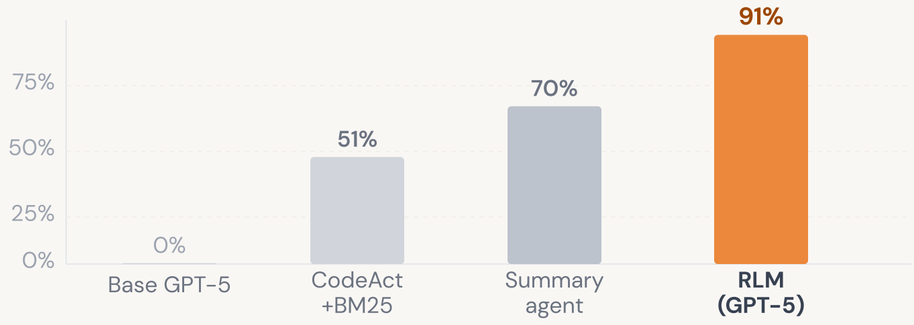
RLM outperforms base GPT-5 by **+28.4%** and base Qwen3-Coder by **+33.3%** — even though the full context fits within the model window.

Root LM is GPT-5; recursive sub-calls use GPT-5-mini. The system selectively routes work — comparable or cheaper median cost than a single GPT-5 call.

RESULTS

At 1,000 Documents, Only the RLM Works at All

BrowseComp-Plus (1K docs · 6–11M tokens) · GPT-5



The setup

1,000 Wikipedia-scale documents — 6–11M tokens total. The base model cannot fit any of these tasks in context at all.

The result

RLM outperforms the summarization baseline by **+29%** and CodeAct+BM25 by **+40%** — at an average cost of $0.99 vs. $1.50–$2.75 for base model ingestion.

The RLM selectively reads context — it doesn't pay to ingest all 6–11M tokens, which is why it's both better *and* cheaper.

RESULTS

RLMs Are a Model-Agnostic Inference Strategy

GPT-5 (COMMERCIAL FRONTIER)

OOLONG	+28.4%
OOLONG-Pairs	0.04 → 58.00 F1
BrowseComp+ (1K)	91.33%

QWEN3-CODER-480B (OPEN FRONTIER)

OOLONG	+33.3%
OOLONG-Pairs	0% → 23.11 F1
CodeQA	strong gains

One fixed system prompt — two very different models

The RLM system prompt is identical across all experiments and is not tuned for any benchmark. The only difference between models is a single extra warning line for Qwen3-Coder about sub-call frequency.

The implication

This is a property of the inference strategy, not a quirk of any particular model. Both models adopt peek-grep-partition-map — and both improve substantially over their base.

The same trajectory CoT followed: a general reasoning behavior that emerges without task-specific engineering.

PART V

Takeaways

TAKEAWAYS

Three Axes of Test-Time Compute

2022 · CHAIN OF THOUGHT

Within a Call

The model thinks longer before answering

2023 · REACT / AGENTS

Across Calls

The model reasons across a chain of tool calls

2025 · RLMS

Over Context

The model decides how to decompose and query its context

Key pattern

Each prior axis became RL-trainable — CoT via RLHF, ReAct via tool-use fine-tuning. There is no reason this axis won't follow the same path.

TAKEAWAYS

RLMs multiply base model progress — they don't replace it.

Today

Frontier LM handles 200k tokens → RLM handles 10M+

Tomorrow

Frontier LM handles 10M tokens → RLM handles 100M+

TAKEAWAYS

Open Problems

End-to-end RL training

Can we train RLMs with outcome-supervised RL like reasoning models? What reward signal captures "used the context well"?

Latency and async

Sequential sub-calls are slow. Async prefix-cached execution would help — but requires new infrastructure. What's the right abstraction?

Depth > 1 recursion

The current implementation stops at depth=1. Deeper trees could handle hierarchical corpora — but how do you control error propagation?

Beyond the REPL

REPL fits structured data well. For unstructured text, audio, or multimodal contexts — what is the right environment for the model to operate in?